

Implementation Calculus of wasm4pm

Exact Formulæ as Implemented in Rust Source

wasm4pm Platform

May 2026

Contents

1	Reinforcement Learning	3
1.1	Reward Function	3
1.2	Health State	3
1.3	Q-Learning (Off-Policy TD)	3
1.4	SARSA (On-Policy TD)	3
1.5	Double Q-Learning (Bias-Corrected)	4
1.6	Expected SARSA	4
1.7	REINFORCE (Monte Carlo Policy Gradient)	4
1.8	LinUCB Contextual Bandit (Sherman–Morrison)	4
2	Statistical Process Control	5
2.1	Western Electric Rules	5
2.2	Process Capability Indices	5
2.3	Normal CDF (Abramowitz & Stegun Approximation)	6
2.4	EWMA	6
3	Prediction Subsystem	6
3.1	N-gram Log Probability	6
3.2	Prediction Confidence & Entropy	7
3.3	Beam Search	7
3.4	Weibull Remaining Time	7
3.5	Remaining Time Confidence	8
3.6	Jaccard Drift Distance	8
4	Conformance Checking	8
4.1	Token-Based Replay Fitness	8
5	Discovery Algorithms	9
5.1	Genetic Algorithm — Fitness-Proportionate Selection	9
5.2	Ant Colony Optimisation — Pheromone Update	9
6	Machine Learning	9
6.1	Ordinary Least Squares Linear Regression	9
6.2	PCA — 2×2 Analytical Eigendecomposition	10
7	Heuristic Miner Dependency Measure	10
8	Resource Prediction: M/M/1 Queue and UCB1 Bandit	11
8.1	M/M/1 Queue Waiting Time	11

8.2	UCB1 Arm Selection	11
8.3	Intervention Ranking Score	11
9	AutoML Configuration Selection	12
9.1	Hardcoded Rule-Based Preset Selection (<code>suggestPreset</code>)	12
9.2	Benchmark-Driven Scoring (<code>suggestPresetFromBenchmarks</code>)	12
9.3	Optimal Config Generation (<code>generateOptimalConfig</code>)	12
10	Summary of Constants	12

Notation

All symbols correspond directly to Rust variable names in the source. Types are given in parentheses where disambiguation matters. File paths are cited at each section head. No background theory is included; every formula is the verbatim implementation.

1 Reinforcement Learning

1.1 Reward Function

File: wasm4pm/src/rl_orchestrator.rs, lines 134–174.

Signature: `compute_reward(prev_health, curr_health, spc_alert_count, guard_pass, circuit_allowed, latency_budget_exceeded)`

$$R = \underbrace{\begin{cases} +1.0 & \text{if } h' < h \\ +0.2 & \text{if } h' = h \\ -1.0 & \text{if } h' > h \end{cases}}_{\text{health component}} + \begin{cases} +0.1 & \text{if } \text{guard_pass} \wedge \text{circuit_allowed} \\ -0.5 & \text{otherwise} \end{cases} - 0.3 \cdot \text{latency_budget_exceeded} \quad (1)$$

where $h = \text{prev_health} \in \{0, \dots, 4\}$ (u8) and $h' = \text{curr_health} \in \{0, \dots, 4\}$ (u8). Reward bounds: $R \in [-5.3, +1.1]$.

1.2 Health State

File: wasm4pm/src/rl_orchestrator.rs, lines 108–120.

Variables: `event_count` (u64), `trace_count` (u64), `unique_activities` (u64).

$$h = \begin{cases} 4 & \text{if } \text{event_count} = 0 \vee \text{unique_activities} = 0 \\ 3 & \text{if } \text{trace_count} = 0 \\ 2 & \text{if } \text{unique_activities} \leq 1 \wedge \text{event_count} < 5 \\ 1 & \text{if } \text{unique_activities} \leq 2 \wedge \text{event_count} < 20 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

1.3 Q-Learning (Off-Policy TD)

File: wasm4pm/src/reinforcement.rs, lines 160–189.

Variables: $\alpha = \text{learning_rate}$ (f32, default 0.1), $\gamma = \text{discount_factor}$ (f32, default 0.99).

$$\text{max_next_q} = \begin{cases} 0 & \text{if done} \\ \max_{a'} Q(s', a') & \text{otherwise} \end{cases} \quad (3)$$

$$\boxed{Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \text{max_next_q} - Q(s, a) \right]} \quad (4)$$

1.4 SARSA (On-Policy TD)

File: wasm4pm/src/reinforcement.rs, lines 341–362.

Identical structure to Q-Learning except $\text{next_q} = Q(s', a')$ where a' is the *actual* next action taken.

$$\boxed{Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right]} \quad (5)$$

1.5 Double Q-Learning (Bias-Corrected)

File: wasm4pm/src/reinforcement.rs, lines 567–618.

Variables: q_a, q_b (separate tables), bootstrap (f32), update_a_first (bool).

A Bernoulli coin-flip with $p = 0.5$ selects which table to update:

$$\begin{aligned} \text{With prob. 0.5: } \text{bootstrap} &= Q_B(s', \arg \max_{a'} Q_A(s', a')) \\ Q_A(s, a) &\leftarrow Q_A(s, a) + \alpha \left[r + \gamma \text{bootstrap} - Q_A(s, a) \right] \end{aligned} \quad (6)$$

$$\begin{aligned} \text{With prob. 0.5: } \text{bootstrap} &= Q_A(s', \arg \max_{a'} Q_B(s', a')) \\ Q_B(s, a) &\leftarrow Q_B(s, a) + \alpha \left[r + \gamma \text{bootstrap} - Q_B(s, a) \right] \end{aligned} \quad (7)$$

1.6 Expected SARSA

File: wasm4pm/src/reinforcement.rs, lines 783–814.

Variables: expected_next (f32), max_q, sum_q, eps = ε , n = $|A|$.

$$\text{expected_next} = \begin{cases} 0 & \text{if done} \\ (1 - \varepsilon) \max_{a'} Q(s', a') + \frac{\varepsilon}{|A|} \sum_{a'} Q(s', a') & \text{otherwise} \end{cases} \quad (8)$$

$$\boxed{Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \mathbb{E}_\pi[Q(s', a')] - Q(s, a) \right]} \quad (9)$$

1.7 REINFORCE (Monte Carlo Policy Gradient)

File: wasm4pm/src/reinforcement.rs, lines 995–1048.

Variables: theta (HashMap<S, Vec<f32>>), g_t (f32), returns (Vec<f32>), pi_j (f32).

Return computation (backward pass):

$$G_t = r_t + \gamma G_{t+1}, \quad G_T = 0 \quad (10)$$

Softmax policy:

$$\pi(a \mid s) = \frac{\exp(\theta_{s,a} - \max_{a''} \theta_{s,a''})}{\sum_j \exp(\theta_{s,j} - \max_{a''} \theta_{s,a''})} \quad (11)$$

Weight update (log-softmax gradient):

$$\theta_{s,a} \leftarrow \theta_{s,a} + \alpha G_t (1 - \pi(a \mid s)) \quad (12)$$

$$\theta_{s,j \neq a} \leftarrow \theta_{s,j} - \alpha G_t \pi(j \mid s) \quad (13)$$

1.8 LinUCB Contextual Bandit (Sherman–Morrison)

File: wasm4pm/src/ml/linucb.rs, lines 130–254.

Variables: $x \in \mathbb{R}^8$ (feature vector), $W \in \mathbb{R}^{5 \times 8}$ (w), $b \in \mathbb{R}^5$ (b), $A \in \mathbb{R}^{8 \times 8}$ (a), A^{-1} (a_inv), $\alpha_{\text{exp}} = \sqrt{2}$ (alpha), $\alpha_{\text{lr}} = 0.1$ (alpha_lr).

Action selection:

$$\text{ucb_bonus} = \alpha_{\text{exp}} \sqrt{x^\top A^{-1} x} \quad (14)$$

$$a^* = \arg \max_a \left(W_a \cdot x + b_a + \text{ucb_bonus} \right) \quad (15)$$

Weight update:

$$\delta = r - (W_{a^*} \cdot x + b_{a^*}) \quad (16)$$

$$W_{a^*} \leftarrow W_{a^*} + \alpha_{\text{lr}} \delta x \quad (17)$$

$$b_{a^*} \leftarrow b_{a^*} + \alpha_{\text{lr}} \delta \quad (18)$$

Covariance update (rank-1 Sherman–Morrison):

$$u = A^{-1} x \quad (19)$$

$$\text{denom} = 1 + x^\top u \quad (20)$$

$$A^{-1} \leftarrow A^{-1} - \frac{u u^\top}{\text{denom}} \quad (21)$$

$$A \leftarrow A + x x^\top \quad (22)$$

2 Statistical Process Control

2.1 Western Electric Rules

File: wasm4pm/src/spc.rs, lines 78–139.

Variables: value (f64), ucl, lcl, cl (f64), recent (&[ChartData]).

Rule 1: value > ucl $\vee$ value < lcl (Out of Control)

Rule 2: $\bigwedge_{i=0}^8 \text{recent}[i].\text{value} > \text{cl} \vee \bigwedge_{i=0}^8 \text{recent}[i].\text{value} < \text{cl}$ (Shift)

Rule 3: $\bigwedge_{i=0}^4 \text{recent}[i+1].\text{value} > \text{recent}[i].\text{value} \vee \bigwedge_{i=0}^4 \text{recent}[i+1].\text{value} < \text{recent}[i].\text{value}$ (Trend)

2.2 Process Capability Indices

File: wasm4pm/src/spc.rs, lines 187–243.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (23)$$

$$s = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n-1}} \quad (24)$$

$$C_p = \frac{\text{USL} - \text{LSL}}{6s} \quad (25)$$

$$C_{pk} = \min\left(\frac{\text{USL} - \bar{x}}{3s}, \frac{\bar{x} - \text{LSL}}{3s}\right) \quad (26)$$

$$z_U = \frac{\text{USL} - \bar{x}}{s}, \quad z_L = \frac{\text{LSL} - \bar{x}}{s} \quad (27)$$

$$P_{\text{defective}} = [1 - \Phi(z_U)] + \Phi(z_L) \quad (28)$$

$$\text{DPMO} = P_{\text{defective}} \times 10^6 \quad (29)$$

$$\sigma_{\text{level}} = \Phi^{-1}(1 - P_{\text{defective}}) + 1.5 \quad (30)$$

2.3 Normal CDF (Abramowitz & Stegun Approximation)

File: wasm4pm/src/spc.rs, lines 274–286.

Variables: t, d, z (f64).

$$t = \frac{1}{1 + 0.2316419 |z|} \quad (31)$$

$$d = 0.39894228 e^{-z^2/2} \quad (32)$$

$$\tilde{\Phi}(z) = 1 - d \cdot t \left(0.319381530 + t(-0.356563782 + t(1.781477937 + t(-1.821255978 + 1.330274429 t))) \right) \quad (33)$$

$$\Phi(z) = \begin{cases} \tilde{\Phi}(z) & z > 0 \\ 1 - \tilde{\Phi}(-z) & z \leq 0 \end{cases} \quad (34)$$

2.4 EWMA

File: wasm4pm/src/prediction_drift.rs, lines 97–110.

Variables: alpha = $\alpha \in (0, 1]$ (f64), smoothed (Vec<f64>), prev (f64).

$$s_i = \begin{cases} x_0 & i = 0 \\ \alpha x_i + (1 - \alpha) s_{i-1} & i \geq 1 \end{cases} \quad (35)$$

α is clamped to [f64::MIN_POSITIVE, 1.0] before use.

3 Prediction Subsystem

3.1 N-gram Log Probability

File: wasm4pm/src/prediction.rs, lines 116–145.

Variables: log_prob (f64), prefix (Vec<String>), prob (f64).

Let $A = (a_0, a_1, \dots, a_T)$ be an activity sequence and n the n -gram order. For each position i :

$$\text{prefix}_i = (a_{i-n+1}, \dots, a_{i-1}) \quad (\text{context window of length } n-1) \quad (36)$$

$$P(a_i \mid \text{prefix}_i) = \frac{\text{counts}[\text{prefix}_i][a_i]}{\sum_a \text{counts}[\text{prefix}_i][a]} \quad (37)$$

$$\text{log_prob} = \sum_{i=1}^T \ln P(a_i \mid \text{prefix}_i) \quad (38)$$

3.2 Prediction Confidence & Entropy

File: wasm4pm/src/prediction_next_activity.rs, lines 44–74.

Variables: confidence, entropy_val, ent, max_ent (f64).

$$\text{confidence} = p_1 \quad (\text{probability of top-1 candidate}) \quad (39)$$

$$H = - \sum_i p_i \ln p_i \quad (40)$$

$$H_{\max} = \ln |\text{candidates}| \quad (41)$$

$$\text{normalized_entropy} = \begin{cases} H/H_{\max} & H_{\max} > 0 \\ 0 & \text{otherwise} \end{cases} \quad (42)$$

3.3 Beam Search

File: wasm4pm/src/prediction_next_activity.rs, lines 141–191.

Variables: beams (Vec<(Vec<String>, f64)>), new_prob (f64), beam_width.

At each step, the beam set $\mathcal{B}$ evolves as:

$$\mathcal{B}_{t+1} = \text{top-}w \left\{ (\sigma \oplus a, p_\sigma \cdot P(a \mid \sigma)) \mid (\sigma, p_\sigma) \in \mathcal{B}_t, a \in \mathcal{A} \right\} \quad (43)$$

where $w = \text{beam_width}$, $\oplus$ denotes sequence concatenation, and beams are ranked by p_σ descending.

3.4 Weibull Remaining Time

File: wasm4pm/src/prediction_remaining_time.rs, lines 70–112.

Variables: cv (coefficient of variation, f64), k (shape), lambda (scale), t (elapsed), cumulative_hazard, hazard_rate, survival, median_remaining (f64).

Parameter estimation from empirical moments:

$$k = \text{cv}^{-1.086} \quad (44)$$

$$\lambda = \frac{\bar{d}}{\Gamma(1 + 1/k)} \quad (45)$$

where $\bar{d}$ is the empirical mean duration.

Hazard, survival, and prediction at elapsed time t :

$$h(t) = \frac{k}{\lambda} \left(\frac{t}{\lambda} \right)^{k-1} \quad (46)$$

$$S(t) = \exp \left[- \left(\frac{t}{\lambda} \right)^k \right] \quad (47)$$

$$H(t) = \left(\frac{t}{\lambda} \right)^k \quad (48)$$

$$\text{median_remaining} = \lambda (H(t) + \ln 2)^{1/k} - t \quad (49)$$

3.5 Remaining Time Confidence

File: wasm4pm/src/prediction_remaining_time.rs, lines 556–580.

Variables: size_factor, bucket_cv, global_cv, precision_factor, confidence (f64).

$$\text{size_factor} = \frac{\text{count}}{\text{count} + 10} \quad (50)$$

$$\text{precision_factor} = \max \left(0, \min \left(1, 1 - \frac{\text{bucket_cv}}{\text{global_cv}} \right) \right) \quad (51)$$

$$\text{confidence} = \min(0.6 \cdot \text{size_factor} + 0.4 \cdot \text{precision_factor}, 0.99) \quad (52)$$

3.6 Jaccard Drift Distance

File: wasm4pm/src/prediction_drift.rs, lines 82–89.

Variables: a, b (HashSet<String>), union (usize), inter (usize).

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (53)$$

$$d_J(A, B) = 1 - J(A, B), \quad d_J(\emptyset, \emptyset) := 0 \quad (54)$$

4 Conformance Checking

4.1 Token-Based Replay Fitness

File: wasm4pm/src/conformance.rs, lines 195–203.

Variables: c (consumed), p (produced), m (missing), r (remaining) (f64).

$$c = \max(\text{consumed_tokens}, 1) \quad (55)$$

$$p = \max(\text{produced_tokens}, 1) \quad (56)$$

$$f_{\text{trace}} = 0.5 \left(1 - \frac{m}{c} \right) + 0.5 \left(1 - \frac{r}{p} \right) \quad (57)$$

$$f_{\text{trace}} = \text{clamp}(f_{\text{trace}}, 0.0, 1.0) \quad (58)$$

This is a symmetric weighted split of the Van der Aalst (2004) fitness formula, with equal weight on the consumption and production components.

5 Discovery Algorithms

5.1 Genetic Algorithm — Fitness-Proportionate Selection

File: wasm4pm/src/genetic_discovery.rs, lines 367–396.

Variables: `total` (f64), `threshold` (f64), `fitness` per individual.

$$\text{total} = \sum_i f_i \quad (59)$$

$$\text{threshold} = U[0, 1) \times \text{total} \quad (60)$$

$$\text{select } j = \min \left\{ j \mid \sum_{i=1}^j f_i \geq \text{threshold} \right\} \quad (61)$$

5.2 Ant Colony Optimisation — Pheromone Update

File: wasm4pm/src/genetic_discovery.rs, lines 450–525.

Variables: `tau` (pheromone), `eta` (heuristic, normalised edge frequency), `alpha`= 1.0, `beta`= 2.0, `evaporation_rate`= 0.1, `q`= 100.0, `deposit` (f64).

Edge selection probability:

$$P(e) \propto \tau_e^\alpha \eta_e^\beta \quad (\alpha = 1, \beta = 2) \quad (62)$$

Evaporation:

$$\tau_e \leftarrow (1 - \rho) \tau_e, \quad \rho = 0.1 \quad (63)$$

Deposit:

$$\text{deposit} = q \cdot f_{\text{ant}} \quad (64)$$

$$\tau_e \leftarrow \tau_e + \text{deposit} \quad \forall e \in \text{ant path} \quad (65)$$

where $q = 100$ and f_{ant} is the fitness of the ant's discovered model.

6 Machine Learning

6.1 Ordinary Least Squares Linear Regression

File: wasm4pm/src/ml/regression.rs, lines 32–108.

Variables: `sum_x`, `sum_y`, `sum_xy`, `sum_xx`, `sum_yy` (f64), `n` (usize).

$$\text{denom}_x = n \cdot \sum x_i^2 - \left(\sum x_i \right)^2 \quad (66)$$

$$\text{denom}_y = n \cdot \sum y_i^2 - \left(\sum y_i \right)^2 \quad (67)$$

$$\text{slope} = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{\text{denom}_x} \quad (68)$$

$$\text{intercept} = \frac{\sum y_i - \text{slope} \cdot \sum x_i}{n} \quad (69)$$

$$R^2 = \frac{(n \sum x_i y_i - \sum x_i \sum y_i)^2}{\text{denom}_x \cdot \text{denom}_y} \quad (70)$$

6.2 PCA — 2×2 Analytical Eigendecomposition

File: wasm4pm/src/ml/pca.rs, lines 54–149.

Variables: mean_x, mean_y, cov_00, cov_01, cov_11 (f64), lambda1, lambda2, discriminant, explained_variance ([f64;2]).

Covariance matrix:

$$\Sigma = \frac{1}{n-1} \begin{pmatrix} \sum (x_i - \bar{x})^2 & \sum (x_i - \bar{x})(y_i - \bar{y}) \\ \sum (x_i - \bar{x})(y_i - \bar{y}) & \sum (y_i - \bar{y})^2 \end{pmatrix} = \begin{pmatrix} \text{cov_00} & \text{cov_01} \\ \text{cov_01} & \text{cov_11} \end{pmatrix} \quad (71)$$

Eigenvalues (characteristic polynomial):

$$\text{tr}(\Sigma) = \text{cov_00} + \text{cov_11} \quad (72)$$

$$\det(\Sigma) = \text{cov_00} \cdot \text{cov_11} - \text{cov_01}^2 \quad (73)$$

$$\text{discriminant} = \max\left(0, \frac{\text{tr}(\Sigma)^2}{4} - \det(\Sigma)\right) \quad (74)$$

$$\lambda_{1,2} = \frac{\text{tr}(\Sigma)}{2} \pm \sqrt{\text{discriminant}} \quad (75)$$

Explained variance:

$$v_k = \frac{\lambda_k}{\lambda_1 + \lambda_2}, \quad k \in \{1, 2\} \quad (76)$$

7 Heuristic Miner Dependency Measure

The Heuristic Miner filters DFG edges by a dependency measure that balances forward and reverse follow counts. For activity pair (a, b) :

Follows and precedes counts (single-pass):

$$|a \rightarrow b| = \text{follows}[(a, b)] \quad (77)$$

$$|b \rightarrow a| = \text{precedes}[(b, a)] \quad (78)$$

Dependency measure:

$$\text{dep}(a, b) = \frac{|a \rightarrow b| - |b \rightarrow a|}{|a \rightarrow b| + |b \rightarrow a| + 1} \quad (79)$$

The +1 in the denominator makes division unconditionally safe (no branch needed), as noted in the source comment. The edge $(a \rightarrow b)$ is retained if and only if

$$\text{dep}(a, b) \geq \theta \quad (80)$$

where $\theta \in (-1, 1)$ is the caller-supplied `dependency_threshold`.

8 Resource Prediction: M/M/1 Queue and UCB1 Bandit

8.1 M/M/1 Queue Waiting Time

Given arrival rate λ (`arrival_rate`) and service rate μ (`service_rate`):

$$\rho = \frac{\lambda}{\mu} \quad (\text{utilization})$$

$$W = \frac{1/\mu}{1 - \rho}, \quad \rho < 1 \quad (\text{mean waiting time})$$

When $\rho \geq 1$ the queue is unstable; the implementation clamps ρ to 0.99 before applying the formula.

8.2 UCB1 Arm Selection

For n resource arms indexed $i = 1, \dots, n$, with empirical mean reward $\hat{\mu}_i$, pull count n_i , and total pulls $N = \sum_j n_j$:

Forced exploration: If any arm satisfies $n_i = 0$, that arm is selected immediately (no score computed).

UCB1 index:

$$\text{ucb}_i = \hat{\mu}_i + c \sqrt{\frac{\ln N}{n_i}}, \quad c = \sqrt{2} \quad (81)$$

The arm with the largest ucb_i is recommended.

8.3 Intervention Ranking Score

Interventions are scored by a convex combination of utility and time-decaying exploration bonus. For the i -th candidate intervention (0-indexed):

$$s_i = e_w \cdot u_i + (1 - e_w) \cdot \min\left(\frac{1}{\sqrt{i+1}}, 1\right) \quad (82)$$

where u_i is the utility score and $e_w \in [0, 1]$ is the `exploration_weight` parameter controlling the exploit/explore balance.

9 AutoML Configuration Selection

9.1 Hardcoded Rule-Based Preset Selection (`suggestPreset`)

Given a set of constraints $C = (\text{maxMemoryMb}, \text{maxLatencyMs}, \text{requiredAlgorithms})$:

1. If `maxMemoryMb` < 1000 $\Rightarrow$ `fast`
2. Else if `requiredAlgorithms` contains `genetic.algorithm` or `ilp` $\Rightarrow$ `quality`
3. Else if `maxLatencyMs` < 200 $\Rightarrow$ `fast`
4. Else if `maxLatencyMs` $\leq$ 30000 $\Rightarrow$ `balanced`
5. Default $\Rightarrow$ `balanced`

9.2 Benchmark-Driven Scoring (`suggestPresetFromBenchmarks`)

For each candidate algorithm a with quality score $q_a \in [0, 100]$ and speed score $v_a \in [0, 100]$ (higher speed score = slower):

Composite score:

$$S_a = 0.6 q_a + 0.4 (100 - v_a) \quad (83)$$

Latency filter: Let τ_a be the benchmark median latency in ms per 100 events (`median_ms_per_100_events`), and L be the caller-supplied log size hint (`logSizeHint`). The algorithm passes the filter if and only if

$$\hat{t}_a = \tau_a \cdot \frac{L}{100} \leq T_{\max} \quad (84)$$

where $T_{\max}$ is `maxLatencyMs`. Algorithms failing the filter are excluded from the candidate set.

Profile compatibility filter: Algorithms are further restricted to those whose `deploymentProfiles` array includes the target `DeploymentProfile`. If no algorithm survives all filters, `suggestPresetFromBenchmarks` falls back to the plain `suggestPreset` rules.

Speed-score to preset mapping: The selected algorithm's v_a determines the returned preset:

$$\text{preset} = \begin{cases} \text{fast} & v_a \leq 30 \\ \text{balanced} & 30 < v_a \leq 55 \\ \text{quality} & v_a > 55 \end{cases} \quad (85)$$

9.3 Optimal Config Generation (`generateOptimalConfig`)

`generateOptimalConfig` chains the above into a single pass:

1. Run `suggestPresetFromBenchmarks` to select preset p^* and top-scoring algorithm a^* .
2. Retrieve base config $\mathbf{c}_{p^*}$ from `getPresetConfig(p*)`.
3. Override `algorithm.name` $\leftarrow a^*$, `algorithm.parameters` $\leftarrow$ defaults for a^* .
4. Return $(\mathbf{c}_{p^*}, a^*, p^*, S_{a^*})$.

10 Summary of Constants

Symbol	Value	Location
α (Q-Learning)	0.1	reinforcement.rs
γ (discount)	0.99	reinforcement.rs
ε (exploration)	0.1	reinforcement.rs
α_{exp} (LinUCB)	$\sqrt{2}$	ml/linucb.rs
α_{lr} (LinUCB)	0.1	ml/linucb.rs
λ_{reg} (LinUCB)	1.0	ml/linucb.rs
SPC Rule 2 window	9	spc.rs
SPC Rule 3 window	6	spc.rs
SPC alert penalty	0.3 per alert	rl_orchestrator.rs
SPC cap	1.5	rl_orchestrator.rs
ACO α	1.0	genetic_discovery.rs
ACO β	2.0	genetic_discovery.rs
ACO evaporation ρ	0.1	genetic_discovery.rs
ACO deposit q	100.0	genetic_discovery.rs
Weibull shape exponent	-1.086	prediction_remaining_time.rs
Confidence size saturation	10	prediction_remaining_time.rs
Confidence floor	0.6 (size), 0.4 (precision)	prediction_remaining_time.rs
Normal CDF p_1	0.2316419	spc.rs
Normal CDF p_2	0.319381530	spc.rs
Heuristic dep. denom. offset	1.0	advanced_algorithms.rs
UCB1 exploration c	$\sqrt{2}$	prediction_resource.rs
M/M/1 stability cap ρ	0.99	prediction_resource.rs
AutoML quality weight	0.6	presets.ts
AutoML speed weight	0.4	presets.ts
AutoML fast speed ceiling	30	presets.ts
AutoML balanced speed ceiling	55	presets.ts

Table 1: Hardcoded constants in the wasm4pm Rust implementation.